JobTrace

A plain-language guide to how your tracker works

This guide explains, in everyday terms, what JobTrace does, where your information is kept, and how it checks your email automatically. It covers both the Windows and macOS versions, and the optional automatic-sync setup that uses your own account keys. No technical background is needed to read it.

Prepared for personal use · August 2026

1 What JobTrace Is

JobTrace is a personal job-application tracker that lives entirely on your own computer — Windows or Mac. Think of it as a private notebook that keeps a tidy record of every job you've applied to — the company, the role, what stage things are at, and everything that has happened along the way.

Nothing about it depends on the internet or on any outside company. Your information is never uploaded anywhere, stored on someone else's server, or shared with a third party. It stays on this computer, in a folder you can open and look at any time.

The two parts working together

– **A dashboard you look at.** A clean, easy-to-read screen that shows your applications, how many you've heard back from, and simple charts of your progress.

– **A helper that reads your email.** A background process that checks your inbox every so often, recognizes recruitment-related emails, and quietly updates your tracker so you don't have to type everything in by hand.

2 Where Your Information Lives

No cloud, no company, no subscription

Everything JobTrace knows is stored in one place: a single data file kept in a folder on this computer, alongside the program itself. There is no external database, no cloud account, and no company on the other end that could see or lose your information.

> **Why keeping it local matters**
> Because everything stays in one file on one computer, there's no risk of two different copies of your data drifting apart or overwriting each other — the kind of problem that can happen with cloud-synced or shared documents. There is exactly one, current, trustworthy copy of your data at all times.

A quick tour of what's kept where

– **Your applications.** Every company, role, current stage, and outcome you track — the heart of the system.

– **Your timeline.** A running history for each application: when you applied, when you heard back, every interview, every update.

– **A record of email checks.** A simple note of when the automatic email check last ran, and a short activity log each time it runs — so you can always see exactly what it found.

– **A “needs your attention” list.** Emails the automatic check wasn't fully confident about are set aside here rather than guessed at, so you can review them yourself.

– **Automatic backups.** Whenever the program needs to upgrade how it stores information, it saves a safety copy first, without being asked.

3 Understanding Your Applications

Every application you track has two separate pieces of information, kept apart on purpose, so that a rejection never erases how far you actually got.

What it tracks	In plain terms
Stage	Where the application currently sits — Applied, Screening, Assessment, an interview round, Offer, or Closed.
Outcome	How it's going — Pending, a positive signal, a rejection, an accepted offer, or one you withdrew from yourself.

For example, an application that reached the final interview and was then rejected still shows “Final Interview” in its history — the rejection doesn't erase how far it got, it's simply recorded as the outcome.

What “heard back” means

The dashboard's “Heard Back” figure counts anything beyond a plain application confirmation — a screening call, an assessment, or an interview invitation all count as having heard back, not just an explicit yes-or-no reply. It's split into positive signals and negative ones, so you can see the balance at a glance.

4 The Dashboard

Opening JobTrace — with the shortcut on your Desktop on Windows, or the Dock/Desktop launcher on a Mac — quietly starts everything in the background and opens the dashboard in your browser. Nothing technical appears on screen; you just see the tracker itself.

What you'll see

– **Summary cards.** Total Applications, Heard Back (broken into positive and negative), Interviews, and Offers — the headline numbers, always visible.

– **The full list.** Every application you're tracking, sortable and searchable, with its current stage and outcome.

– **Charts.** How many applications you've sent over time, a breakdown of where everything currently stands, and a view of your overall journey.

– **An email-status indicator.** A small pill in the top bar shows when the automatic email check last ran and a short summary of what it found.

5 How the Automatic Email Check Works

Read-only, careful, and cautious by design

JobTrace can look through your email for recruitment-related messages and update your tracker on its own. This connection is deliberately narrow: it can only look at your email, and nothing else.

> **It can never take action in your inbox**
> The email check is only ever allowed to search for and open messages. It cannot send an email, write a reply, delete or archive anything, change any label, mark anything as read or unread, or unsubscribe from anything on your behalf. Those abilities simply aren't given to it — there's no setting to accidentally enable them.

What happens during a check

1. It picks up where it left off, only looking at messages received since the last check — it doesn't re-read your whole inbox every time.
2. It looks for recruitment-related messages, including ones you've already moved to Trash, since it's normal to read and then tidy up an email.
3. It works out what each relevant email means — an application confirmation, an interview invitation, a rejection, and so on — and matches it to the right application in your tracker, or creates a new one if it's genuinely new.
4. When it isn't confident about a match, it doesn't guess. It sets the email aside in a “needs your attention” list instead of risking a mistake.
5. It remembers every email it has already looked at, so re-running a check never creates duplicates or repeats an update.
6. It finishes by writing a short, plain-English summary of what it found and changed — saved so you can look back at it any time.

There are actually two different ways this can be powered under the hood — using the Claude Code program directly, or an optional script that talks to Claude on its own, using an account you set up yourself. Day to day they work identically, exactly as described above; Section 14 explains the difference for anyone curious.

6 How Often It Runs

The email check happens automatically on whatever schedule was set up for it (every 6–8 hours is typical), using your computer's own built-in scheduling tool — Task Scheduler on Windows, or launchd on a Mac. It only runs while your computer is on (and, on Windows, while you're signed in); it won't run if the computer is off or asleep.

By default, there's nothing to maintain and nothing to pay for — it uses your existing sign-in to Claude, so there's no separate account, subscription, or ongoing cost involved. If the optional setup described in Section 14 was used instead, a small per-use cost applies — but that only happens if it was deliberately set up that way.

If you ever want to check on it or turn it off

– **To see what a check found,** look at the email-status pill on the dashboard, or open the day's activity log in your JobTrace folder.

– **To pause or stop it on Windows,** open the Task Scheduler tool built into Windows (search for “Task Scheduler” in the Start menu), find the entry for JobTrace's Gmail sync, and disable or delete it.

– **To pause or stop it on a Mac,** run `launchctl unload ~/Library/LaunchAgents/com.jobtrace.gmailsync.plist` in Terminal, or simply delete that file.

Turning the automatic check off never affects your saved applications — it only stops new emails from being read in going forward.

7 How Your Data Is Kept Safe

– **It never leaves this computer.** There's no cloud backup, no syncing service, and no outside company with a copy of your information.

– **Safety copies before any upgrade.** If the way information is stored ever needs to change, a full backup is made automatically first, without being asked.

– **No duplicate entries.** Both the dashboard and the automatic email check are built so the same action can never be recorded twice — even if a check is accidentally run twice over the same emails.

– **You can always export a copy.** The dashboard can save your entire list of applications to a plain spreadsheet-friendly file at any time, giving you an independent copy whenever you'd like one.

8 Quick Reference

Opening and closing JobTrace

– **To open it,** double-click the JobTrace shortcut on your Desktop (Windows), or double-click `start.command` (macOS). It starts quietly in the background and opens the dashboard for you.

– **To close it,** simply close the browser tab — the background helper can be left running, or stopped from the JobTrace folder if you prefer (`stop.bat` on Windows, `stop.command` on macOS).

Checking on the automatic email sync

– **The status pill** at the top of the dashboard always shows the last check's time and a short summary.

– **A written summary** of every check is also saved in your JobTrace folder, if you'd like the fuller picture of what it looked at and found.

– **Anything it wasn't sure about** is listed for your review right in the dashboard — nothing is ever silently guessed.

The sections above are everything you need day to day. What follows is an optional deeper look under the hood — how the program is actually built, what each piece is called, and exactly how the automatic email check talks to Claude. Nothing here requires a technical background; every term is explained as it comes up.

9 Behind the Scenes: The Building Blocks

JobTrace is built from three separate pieces that work together, plus a fourth piece — the automation — that ties into them from the outside. Each one has a plain job to do.

Piece	Its job	Built with
The website (frontend)	What you actually see and click on screen	HTML, CSS, and JavaScript — the same three building blocks every website is made of
The engine (backend)	Does the real work: saves data, runs calculations, enforces the rules	Python, a widely used programming language
The filing cabinet (database)	Stores every application and its history as one file on this computer	SQLite, a simple, file-based way of storing information
The automation	Reads Gmail on a timer and updates the filing cabinet for you	Your computer's own scheduler (Task Scheduler on Windows, launchd on macOS) plus either the Claude Code program, or — optionally — a small script using your own account keys directly

None of this runs on the internet. The "website" only exists on this computer and is never visible to anyone else, even though it looks and behaves like an ordinary website in your browser.

10 The Website You See

The frontend

Everything you look at — the dashboard, the charts, the forms for adding an application — is a small website made of three kinds of files, all stored in JobTrace's frontend folder:

– **HTML (HyperText Markup Language)** lays out what's on the page — this box here, that button there. One file, index.html, defines the entire page.

– **CSS (Cascading Style Sheets)** decides how it all looks — the dark background, the gold highlights, the layout of the cards and charts. This lives in a single styles.css file.

– **JavaScript** makes the page interactive — it's what redraws the charts, opens and closes forms, and talks to the engine behind the scenes without ever needing to reload the page. This is split into several small files by purpose (one for the dashboard, one for charts, one for the Gmail status pill, and so on).

Your browser (Chrome, Edge, Safari, whichever you use) is what actually reads these files and turns them into the screen you interact with — the same way it would for any website, except these files are sitting on your own hard drive instead of being downloaded from the internet.

11 The Program Doing the Work

The backend

Behind the website sits a small Python program. Python is simply the language its instructions are written in — nothing about it is visible to you day to day, and it runs exactly the same way on Windows or a Mac. When you start JobTrace, this program quietly begins running in the background and does two jobs at once:

1. It hands your browser the HTML, CSS, and JavaScript files described above, so the dashboard can appear in the first place.
2. It listens for requests from that page — “give me the list of applications,” “save this new one,” “what are today's statistics” — and answers each one. This back-and-forth only ever happens between your browser and this program, both on the same computer; nothing is sent out to the internet.

Each of those requests goes to a fixed, specific address, such as /api/applications or /api/stats/summary — this fixed list of addresses is usually called an API. Think of it as a very short, very literal menu: the webpage can only ask for exactly the things on that menu, nothing else, which keeps the whole system predictable and easy to reason about.

The program is split into a few files, each with one job

File	What it's responsible for
server.py	Listens for requests from the browser and routes each one to the right place
repository.py	The actual rules — how to create, update, and search applications; how the dashboard numbers are calculated
database.py	Opens and manages the data file itself; sets it up the first time it's used
gmail_sync.py	Keeps track of which emails have already been checked, and when the last email check ran
constants.py	The fixed lists everything else relies on — the valid stages, outcomes, and categories
gmail_api_sync.py	Optional: the standalone script for the alternate, own-account-key automation path described in Section 14 — not used at all unless you set it up

12 Where Your Data Actually Lives

The database

Every application, every stage change, every event on a timeline is stored in one single file: data/applications.db. It uses a format called SQLite — essentially a very organized, structured filing cabinet built into a single file, rather than a spreadsheet or a folder of loose documents. Nothing needs to be installed or run separately for this to work; Python can read and write it directly, on Windows or macOS.

Inside that one file, your information is split into two related tables: one row per application (company, role, current stage, outcome), and one row per event on that application's timeline (an interview invitation, a rejection, and so on) — linked back to the application it belongs to.

> **Why the dashboard and the email check never collide**
> The data file runs in what's called “WAL mode” (write-ahead logging) — a setting that lets one process read the file while another is writing to it, safely, at the same moment. That's what allows you to have the dashboard open in your browser at the exact same time the scheduled email check is quietly updating it in the background, with no risk of the two interfering with each other.

The other files alongside it

– **gmail_sync_state.json,** a small file recording the date and time of the last successful email check.

– **unresolved_gmail_items.json,** the “needs your attention” list of emails the check wasn't confident enough to act on by itself.

– **sync_logs,** a folder holding one plain-text report per email check, so you can look back at exactly what any run found.

– **The backups folder,** which automatically receives a full safety copy of applications.db any time the structure of the data needs to change.

13 From Click to Saved

What actually happens behind the scenes

As a concrete example, here's exactly what happens when you add a new application by hand on the dashboard:

1. You fill in the form in your browser and click Save. The page's JavaScript gathers what you typed.

2. It sends that information, on this computer only, to the Python program's /api/applications address — a request that essentially says “please create this application.”

3. The Python program checks that everything makes sense (a valid stage, a valid outcome, no missing required details), then writes a new row into applications.db.

4. It sends a confirmation back to the page, which then redraws itself to show the new application — all without the page ever reloading.

Every action in JobTrace — editing a stage, adding a timeline event, deleting an application, exporting your data — follows this same basic pattern: the page asks, the Python program checks and carries it out, the data

file is updated, and the page reflects the result.

14 How the Automatic Sync Actually Runs

Two ways to power the automation, step by step

This is the piece that runs without you doing anything, so it's worth spelling out exactly how it works — there's no mystery to it, just a chain of ordinary steps. There are two ways it can be set up; both follow the exact same email-reading rules from Section 5 and write into your tracker the exact same way — they differ only in what's doing the reading.

Which one should you use?

	Default way (Claude sign-in)	Optional way (your own API key)
Cost	Free — covered by your existing Claude subscription	A small charge to your own account for every sync
Setup effort	Low — install one app, approve Gmail once	Higher — a Google developer setup step, plus an API key
Needs an app installed	Yes (Claude Code or Codex)	No — nothing but Python
Best for	Almost everyone	Running it on a computer without Claude installed

> **If you're not sure, use the default way**
> It costs nothing extra, takes less setup, and is what the rest of this guide assumes. Only consider the optional way if you specifically want the automatic check to run on a computer that doesn't have Claude Code or Codex on it.

The default way: your existing Claude sign-in

1. On whatever schedule was set up, your computer's own scheduler — Task Scheduler on Windows, or launchd on a Mac — starts a small script (run_gmail_sync_claude on Windows or macOS, or the Codex CLI equivalent).
2. That script launches the Claude Code program — the same program and the same account sign-in used throughout this project — in a “non-interactive” mode. That simply means it runs once, quietly, with no chat window, and closes itself the moment it's done.
3. It hands that program one specific set of written instructions: the contents of GMAIL_SYNC_TASK_PROMPT.md, a plain-text file that spells out exactly what to look for in your email and exactly how it's allowed to update the tracker.
4. Claude reaches your Gmail account through what's called an MCP connector — a secure, pre-approved bridge that lets it search for and open messages. It never sees or handles your password, and the connector simply doesn't offer any way to send, delete, or label email — those actions aren't available to be used, not just switched off.

5. For every relevant email it finds, Claude uses the same repository.py rules the dashboard itself relies on to safely write the update into applications.db — never by editing the file directly, always through those same guarded, double-checked functions.
6. Before finishing, it writes a plain-English summary of what it did into a new file in data/sync_logs, and updates gmail_sync_state.json with the time of this successful check, so the next run knows exactly where to pick up.
7. The Claude Code program then exits completely. Nothing stays running in the background between checks — the computer goes back to doing nothing related to JobTrace until the next scheduled time.

> **You don't need to be talking to Claude for this to work**
> This whole chain is started directly by your computer itself, on a timer — not by you opening a chat. It reuses the sign-in already set up on this computer, the same way an email app can check for new mail on a schedule without you opening it first.

The optional way: a script using your own account instead

This is not the default, and none of it happens unless it was deliberately set up. JobTrace can instead run a small program of its own — gmail_api_sync.py — that talks to Gmail and to Claude directly, without opening the Claude Code program at all. This exists for one reason: it lets the automatic check run on a computer that doesn't have Claude Code or Codex installed.

Setting it up this way requires two things you'd deliberately create yourself first:

– **A one-time Google sign-in for this program,** similar to approving a new app's access to your Gmail the first time — after that one approval, no browser or sign-in is needed again.

– **An Anthropic API key,** a personal access code from Claude's developer website that lets a program use Claude directly. Unlike the default way, this bills a small, usage-based amount to your own account for each sync — you'd see this because you would have created the key and account yourself, specifically to enable it.

Everything else works exactly the same as the default way: read-only Gmail access, the same matching and update rules from GMAIL_SYNC.md, the same duplicate-protection checks, and the same plain-English log of what it did each time, saved to data/sync_logs. Nothing about your saved applications differs based on which of the two ways produced an update — this only changes what powers the automatic check itself.

About API keys

The default setup described above deliberately avoids a metered API key entirely — it reuses your existing Claude Code sign-in, so there's no separate bill to watch. If the optional way was set up instead, that's a deliberate choice someone made, requiring an account and a key they created on purpose — it's never present by accident. JobTrace's own code (the website and the Python program behind it) never contains an AI model and never calls one on its own; the only two ways any “reading and understanding an email” intelligence ever gets involved are the two described in this section, and both require deliberate setup first.

A quick note on the word “API”

Earlier, this guide mentioned JobTrace's own small API — the short list of web addresses (like /api/applications) that the dashboard's webpage uses to talk to its own Python program on this computer. That's a completely separate, unrelated thing from an AI API: it's purely local, free, and has nothing to do with Claude. Nowhere in JobTrace's own code — not the website, not the Python program — does anything call an external, paid AI API on its own; that only ever happens through one of the two deliberately-configured automation paths described above.